CHRIST UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING / AI & ML

5th Semester - Advanced JavaScript Backend Frameworks (Node.js & Express JS)

Continuous Internal Assessment - 3 (CIA-3) - Final Project Report

Project Code & Title	P03 - Hotel Room Booking & Reservation Platform
Course Name	Advanced JavaScript Backend Frameworks (Node.js & Express JS)
Section / Batch	5BTCSAIML B
Semester	5th Semester (Academic Year 2026-2027)
GitHub Repository	https://github.com/annmary-aaa/hotel-booking-platform

Mandatory Team Details Page

S.No	Student Name	Roll No.	Department	Section / Batch
1	Ann Mary Johnson	2462039	ADSE	5BTCSAIML B
2	Anki Pai	2462036	ADSE	5BTCSAIML B
3	Allen Prem Varghese	2462030	ADSE	5BTCSAIML B
4	Ronit Anegundi	2462188	ADSE	5BTCSAIML B

Submission Verification: Source repository hosted live on GitHub (Public) at **https://github.com/annmary-aaa/hotel-booking-platform**

1. Project Overview

Grand Horizon is a multi-property hotel room booking, reservation, and operations management platform developed for the Continuous Internal Assessment 3 (CIA-3) final assessment in Advanced JavaScript Backend Frameworks. Engineered on the prescribed Node.js, Express.js, and MongoDB stack, the system translates real-world hospitality challenges - real-time room search across properties, strict double-booking concurrency prevention, date-based dynamic yield pricing, role-separated front desk workflows, and aggregation-driven executive occupancy analytics - into an enterprise-grade RESTful service.

Rather than treating room rates as static numbers, the platform implements a decoupled pricing engine: standard base prices are associated with room categories, while seasonal surge windows and weekend multipliers are dynamically evaluated at query and booking time. The application is complemented by a lightweight, responsive client interface featuring an elegant cream/espresso aesthetic with custom vector line art and zero external UI dependencies, providing seamless evaluation for guests, staff, and administrators.

1.1 Problem Statement

Independent and boutique hotel chains frequently struggle with fragmented management tools and spreadsheet-based booking records. This leads to severe operational deficiencies: (a) risk of double-booking under concurrent traffic, (b) inability to automatically apply dynamic weekend or holiday pricing rules, (c) lack of coordination between front-desk check-in operations and housekeeping room cleanliness, and (d) absence of centralized property occupancy and revenue reporting. Grand Horizon resolves these operational bottlenecks by unifying all hotel domain models within an indexed MongoDB database, governed by strict business rules and exposed via role-protected Express endpoints.

1.2 Objectives

- **Self-Service Booking:** Deliver a self-service guest portal enabling live availability search by city, dates, and occupancy without manual staff intervention.
- **Concurrency Control:** Prevent double-booking through an interval-overlap validation algorithm enforcing inventory thresholds before record creation.
- **Dynamic Pricing:** Calculate dynamic nightly rates by combining base prices with active seasonal overlays, weekend multipliers, and 12% statutory tax.
- **Front Desk & Housekeeping:** Provide front-desk staff with a dedicated console to manage confirmed arrivals, enforce clean-room check-in guards, and auto-flag departures to dirty.
- **Security & RBAC:** Implement Role-Based Access Control (RBAC) via cryptographically signed JWT tokens partitioning Guest, Staff, and Admin privileges.
- **Management Analytics:** Expose high-performance MongoDB aggregation pipelines computing property-level occupancy rates and net revenue analytics.
- **Code Quality:** Follow corporate engineering standards: MVC directory structure, Joi input validation, centralized error handling, and an automated 25-test suite.

2. Technology Stack

The platform is built end-to-end in JavaScript, maximizing developmental velocity and code maintainability across the backend API, database models, and client interface during the 12-day sprint.

Layer / Component	Technology Selected	Role and Implementation in Project
Runtime Environment	Node.js (v18+ / v24)	Asynchronous, event-driven JavaScript runtime executing the Express application.
Web Framework	Express.js (v4.19)	RESTful routing, middleware pipelines, and centralized HTTP request-response dispatch.
Persistence Layer	MongoDB Atlas (Cluster0)	Document-oriented cloud database hosting 6 collections with secondary indexing.
Object Data Modeling	Mongoose ODM (v8.3)	Schema definitions, type casting, pre-save hashing hooks, and relationship population.
Authentication & RBAC	JWT + bcryptjs	Stateless JSON Web Tokens with 7-day expiry; bcrypt password hashing with 10 salt rounds.
Request Validation	Joi Schema Validation	Middleware intercepting all incoming request bodies to enforce strict types and reject malformed data.
Automated Testing	Native Node Test Runner	End-to-end integration test suite (test/test.js) validating 25 test cases across all 13 modules.
API Documentation	Postman Collection	postman_collection.json preconfigured with dynamic JWT token capture for all 15 endpoints.
Client Interface	Vanilla HTML5 / CSS3 / JS	Lightweight boutique single-page client with bespoke vector SVG icons and no build step.

3. System Architecture & Engineering Standards

The backend adheres strictly to the Model-View-Controller (MVC) architectural pattern. Express route handlers declare endpoint surfaces, controllers encapsulate business logic, Mongoose models manage schema validation

and MongoDB persistence, and modular middleware layers handle cross-cutting concerns including authentication, role gating, input sanitization, and centralized exception dispatch.

3.1 MVC Project Directory Structure

The codebase is cleanly separated into specialized directories, ensuring that business rules, database queries, and route declarations are strictly isolated:

```
hotel-booking-platform/
├── config/            # MongoDB connection via Mongoose (db.js)
├── controllers/       # 10 modular controllers isolating domain logic
├── middleware/        # auth.js (JWT & RBAC), validate.js (Joi), errorHandler.js
├── models/            # 6 Mongoose schemas with virtuals and indexes
├── routes/            # Express REST route definitions grouped by resource
├── utils/             # Pricing engine, availability overlap, cancellation tiers, seed.js
├── validators/        # Joi request validation schemas
├── test/              # Automated end-to-end integration test suite (25 tests)
├── public/            # Client frontend (Vanilla HTML5/CSS3/JS, SVG iconography)
├── postman_collection.json # Official testing collection with auto-saving token script
└── server.js          # Express application entrypoint and middleware registry
```

3.2 Centralized Error Handling & Standardized HTTP Contracts

All controller actions are wrapped in an asynchronous exception catcher (asyncHandler.js). If a runtime error or business conflict occurs, it is formatted through an ApiError operational error instance and returned via middleware/errorHandler.js. This guarantees uniform JSON responses and completely eliminates unhandled promise rejections or server crashes.

HTTP Status	Error Code	Trigger Condition & System Behavior
400 Bad Request	VALIDATION_ERROR	Joi schema validation fails (e.g. check-out date precedes check-in date).
401 Unauthorized	NO_TOKEN / INVALID_TOKEN	Missing, expired, or cryptographically invalid Bearer JWT token.
403 Forbidden	FORBIDDEN	Authenticated user lacks role privilege (e.g. guest accessing admin reports).
404 Not Found	NOT_FOUND	Provided ObjectId does not correspond to any active record in MongoDB.
409 Conflict	AVAILABILITY_CONFLICT	Reservation attempt when remaining room inventory for requested date range is zero.
409 Conflict	ROOM_NOT_READY	Front desk attempts to check in a guest to a room whose housekeeping status is still dirty.

409 Conflict	INVALID_STATUS_TRANSITION	Illegal booking state progression (e.g. attempting to check in a cancelled booking).

4. Database Design & Schema Architecture

The persistence layer is hosted on MongoDB Atlas Cluster0 under database hotel_booking_platform. The schema partitions static property and inventory metadata from dynamic, time-sensitive transactional records.

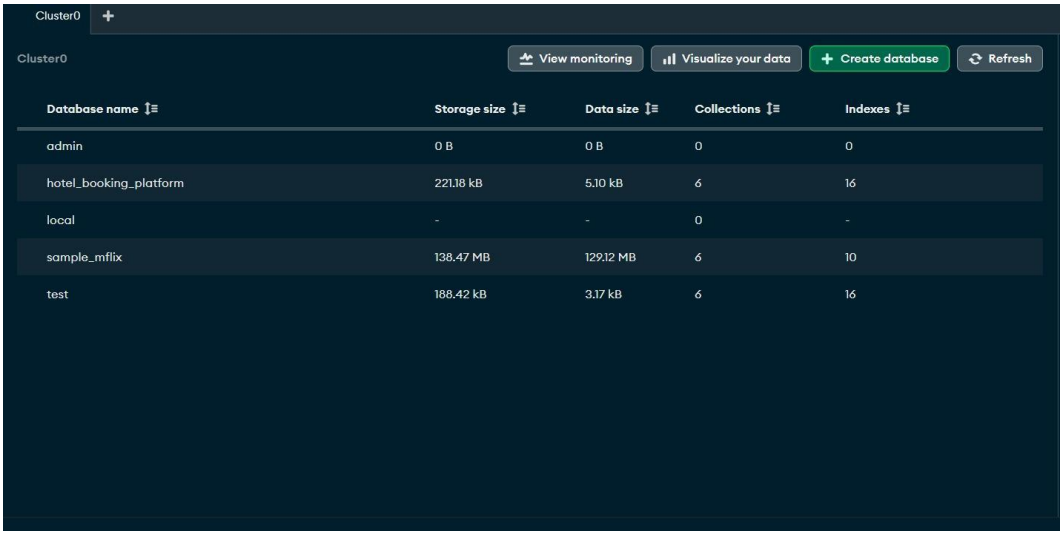

Figure 1 - hotel_booking_platform database: 6 collections, 16 indexes on MongoDB Atlas Cluster0

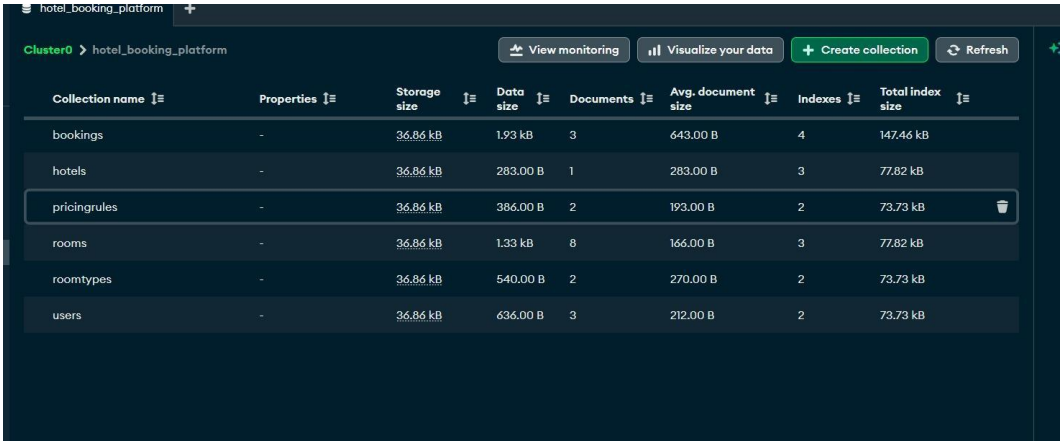

Figure 2 - Collection-level view showing document counts, storage sizing, and index counts

4.1 Entity-Relationship (ER) Architecture

The database models one-to-many and relational references using Mongoose ObjectId linking, maintaining optimal document size while embedding small, frequently read sub-structures such as amenities arrays and audit history:

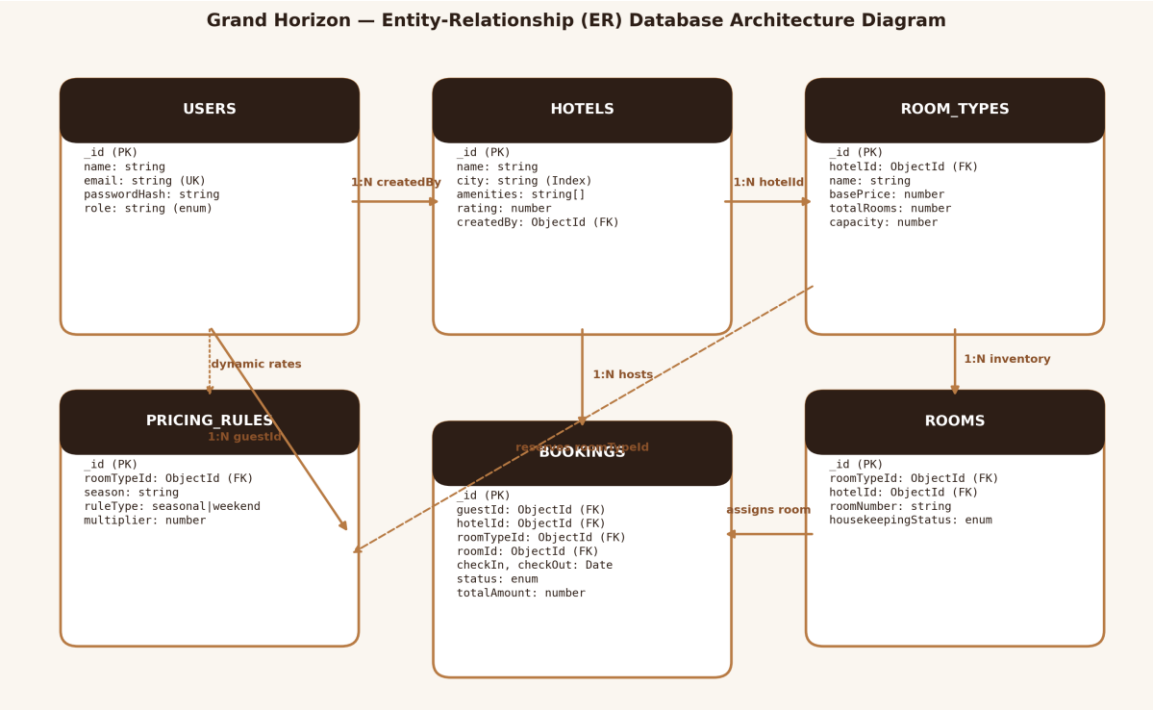

Figure 3 - Grand Horizon Entity-Relationship (ER) Database Architecture Diagram

4.2 Database Indexing Strategy

To guarantee rapid query response times and eliminate full collection scans under concurrent traffic, indexes were deliberately created on high-cardinality keys:

- **Authentication:** users ({ email: 1 }, Unique): Enforces account uniqueness and speeds up login lookups to O(1).
- **Search & Filter:** hotels ({ name: 1 }, { city: 1 }): Powers rapid geographic searches and keyword filtering.
- **Property Inventory:** roomtypes ({ hotelId: 1 }): Accelerates inventory fetching for property detail pages.
- **Room Operations:** rooms ({ roomTypeId: 1 }, { hotelId: 1 }): Speeds up front desk lookups for available clean rooms.
- **Transactional Queries:** bookings ({ guestId: 1 }, { hotelId: 1 }): Optimizes guest reservation history and front desk live queues.
- **Dynamic Pricing:** pricingrules ({ roomTypeId: 1 }): Enables rapid multiplier lookups during dynamic rate computation.

4.3 Collections & Schema Field Design

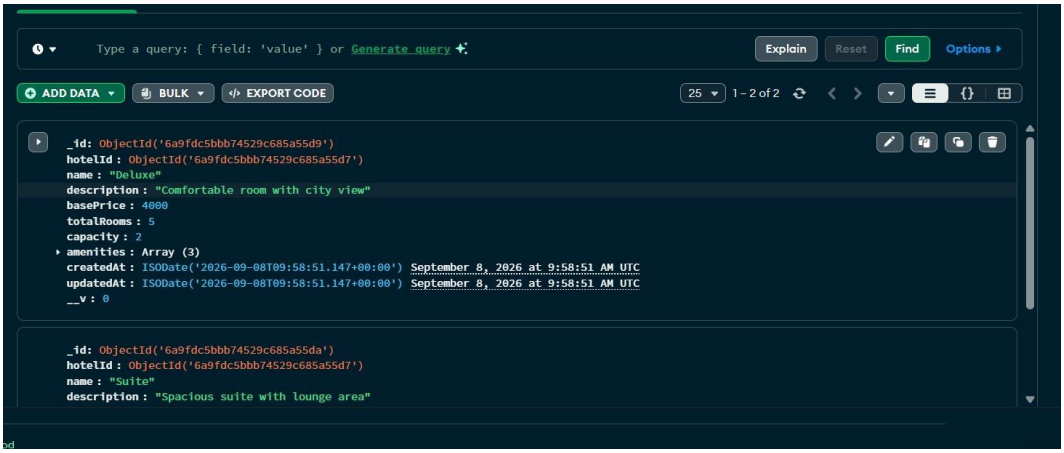

Figure 4 - roomtypes collection: Deluxe and Suite documents with base price, capacity and amenities

5. Functional Modules Specification (13/13 Completed)

In full compliance with Section P03 of the assessment guidelines, all 13 required functional modules have been fully implemented, tested, and connected end-to-end to MongoDB Atlas:

#	**Assessment Module**	**Primary Endpoint(s)**	**Role Access**	**Business Logic & Verification**
1	Guest Registration & Auth	POST /api/auth/register POST /api/auth/login	Public	Bcrypt password hashing (10 salt rounds), signed JWT tokens, role gating.
2	Hotel & Property Mgmt	POST /api/hotels GET /api/hotels	Admin / Public	Multi-property management, embedded amenities, soft-deletion flag.
3	Room Type & Inventory	POST /api/room-types GET /api/rooms	Admin / Staff	Room type pricing, total inventory threshold. Auto-provisions physical rooms.
4	Availability Search Engine	GET /api/hotels/search	Public	Interval overlap algorithm (checkIn < reqEnd && checkOut > reqStart), live rate calculation.
5	Reservation Booking	POST /api/bookings	Guest	Double-booking prevention; real-time inventory locking, dynamic folio persistence.
6	Dynamic Pricing Rules	POST /api/pricing-rules GET /api/pricing-rules	Admin / Public	Multi-tier rule engine: seasonal date windows

				and Friday/Saturday weekend multipliers.
7	Booking Status Management	PUT /api/bookings/:id/confirm	Staff / Admin	State machine: reserved -> confirmed -> checked_in -> checked_out. Rejects invalid leaps.
8	Check-in / Check-out Module	PUT /api/bookings/:id/checkin PUT /.../checkout	Staff / Admin	Check-in verifies clean/inspected room status; check-out auto-flags room to dirty.
9	Housekeeping Status Tracking	PUT /api/rooms/:id/housekeeping	Staff / Admin	Updates cleanliness lifecycle: clean, dirty, in_progress, inspected, out_of_service.
10	Cancellation & Refund Engine	PUT /api/bookings/:id/cancel	Guest / Staff	Tiered refund schedule: >7 days = 100%, 2-7 days = 50%, <48h = 0% forfeiture.
11	Guest Booking History	GET /api/guests/:id/bookings	Guest (Self) / Staff	Segregates upcoming from past bookings. Protected by user ownership validation.
12	Invoice Generation Summary	GET /api/bookings/:id/invoice	Guest / Staff	Itemized folio: room base rate, stay nights, dynamic multiplier, 12% tax, grand total.
13	Admin Occupancy Reports	GET /api/admin/reports/occupancy	Staff / Admin	MongoDB Aggregation Pipeline ($match, $lookup, $group, $project) for occupancy % & revenue.

6. Dynamic Pricing & Folio Calculation Engine

To verify mathematical correctness, the pricing engine was reconciled between database values and client-facing search outputs. For a weekday stay (Wednesday, 9-10 September 2026), Deluxe is quoted at ₹4,480 and Suite at ₹8,960 for one night:

Room Type	**Base Price**	**Multiplier Applied**	**Pre-Tax Subtotal**	**Statutory Tax (12%)**	**Guest-Facing Folio Total**
Deluxe Suite	₹4,000	1.0 (Standard weekday)	₹4,000	₹480	₹4,480 / night

Executive Suite	₹8,000	1.0 (Standard weekday)	₹8,000	₹960	₹8,960 / night

Both rates follow the formula: Total = (BaseRate × Multiplier) × 1.12. On weekend dates (Friday/Saturday nights), the 1.2x weekend multiplier in pricingrules dynamically increases the pre-tax base rate before GST is applied, demonstrating real-time yield management.

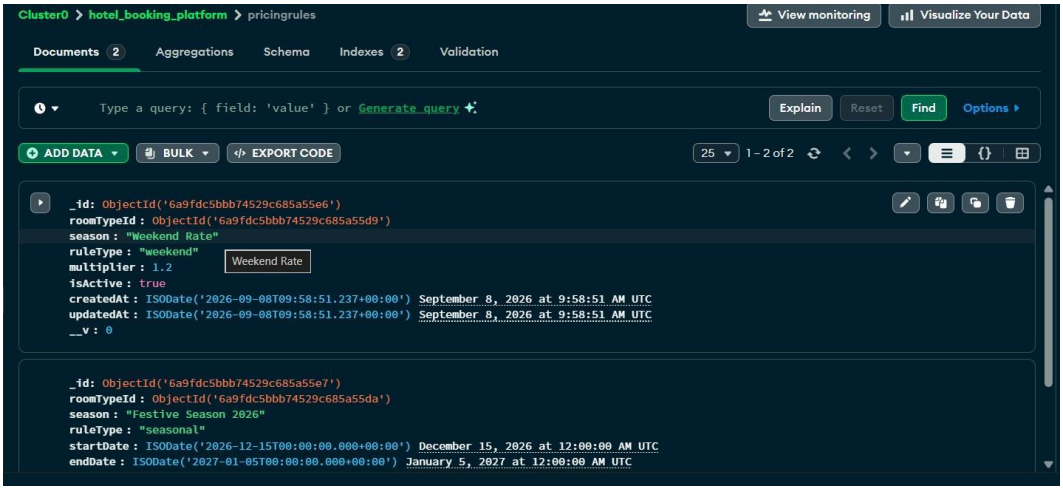

Figure 5 - pricingrules collection: the 1.2x weekend rule and festive seasonal rule with date ranges

7. Guest Portal Walkthrough

The client interface exposes a seamless, self-service guest experience allowing travellers to search real-time inventory, reserve rooms, inspect itemized invoice folios, and manage bookings.

7.1 Real-Time Availability Search

Guests specify a destination city, check-in date, check-out date, and guest count. The search endpoint dynamically evaluates all physical inventory and displays remaining rooms:

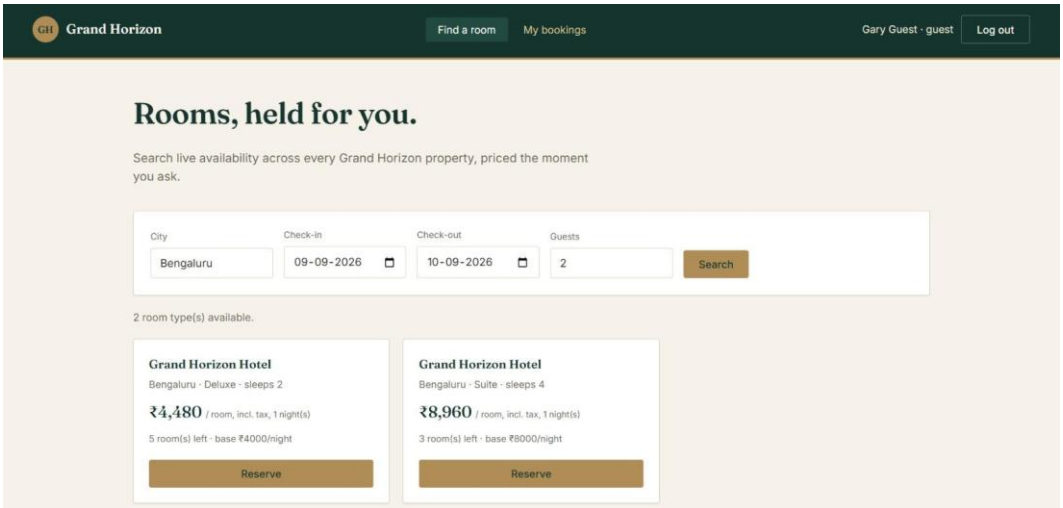

Figure 6 - Search results for Bengaluru: 2 room types available, dynamically priced with tax

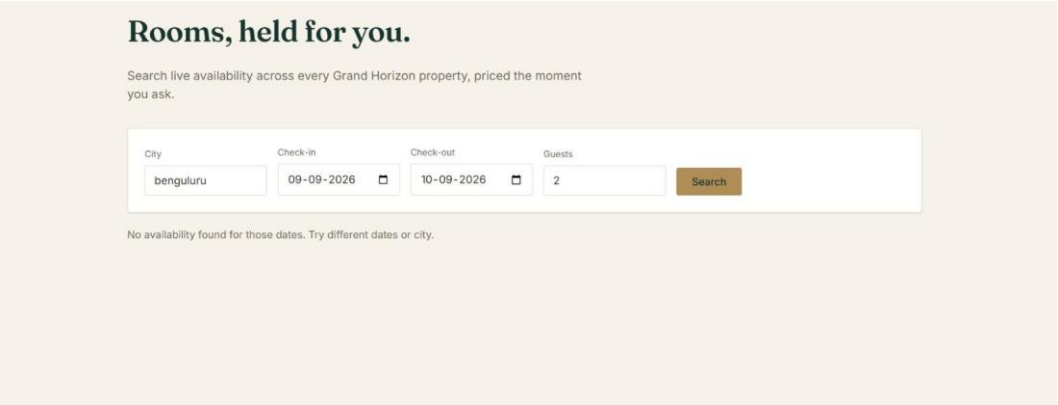

Figure 7 - Search form with unmatched query correctly falling back to friendly 'no availability' message

7.2 Self-Service Reservation & Booking History

Clicking 'Reserve' executes POST /api/bookings under the guest's JWT token. The reservation is persisted in MongoDB and listed under the guest's 'My bookings' portal with real-time cancellation controls:

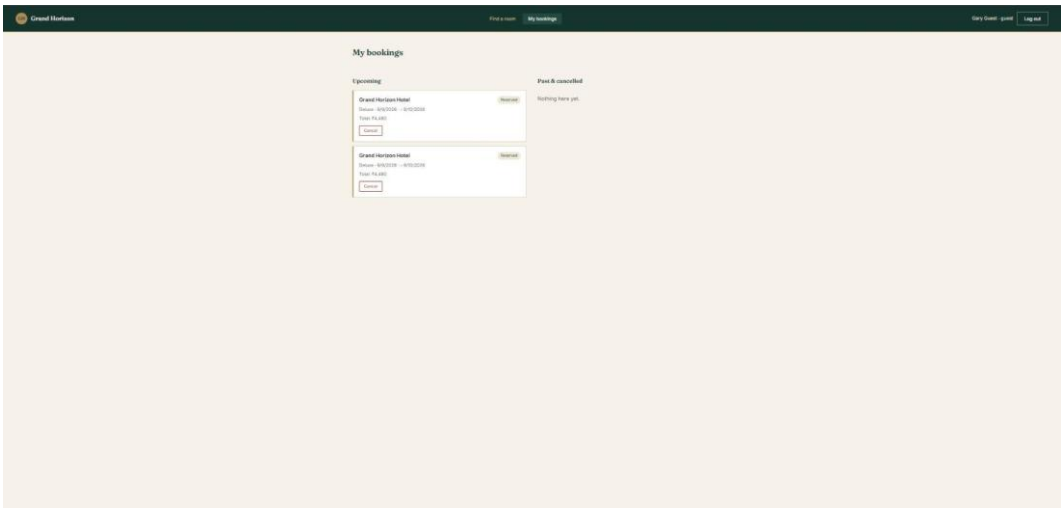

Figure 8 - Guest 'My bookings' portal: active reservations with real-time cancellation controls

8. Staff Operations & Executive Analytics

The staff console consolidates front-desk reservations with physical room housekeeping operations onto a single operational dashboard.

8.1 Front Desk Queue & Housekeeping Management

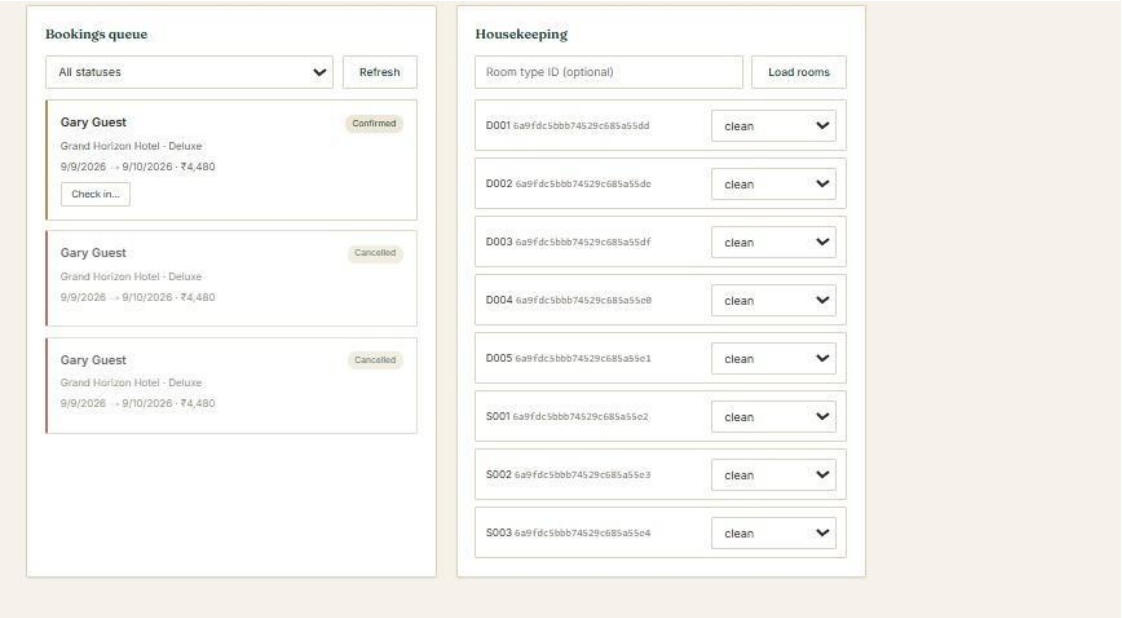

Figure 9 - Staff dashboard: live bookings queue alongside housekeeping status board

8.2 Admin Occupancy & Revenue Analytics (Module 13)

To provide property managers with yield visibility, the backend implements GET /api/admin/reports/occupancy. The endpoint executes a 4-stage MongoDB aggregation pipeline ($match, $lookup, $group, $project) that computes total room-nights booked, total available nights, property occupancy rate percentage, and gross generated revenue.

8.3 Automated End-to-End Test Suite Verification

To prove software robustness, an automated integration test suite (test/test.js) was written using Node's native assert module. Executed via npm test, it programmatically validates all 13 modules, RBAC security gates, and concurrency edge cases across 25 tests:

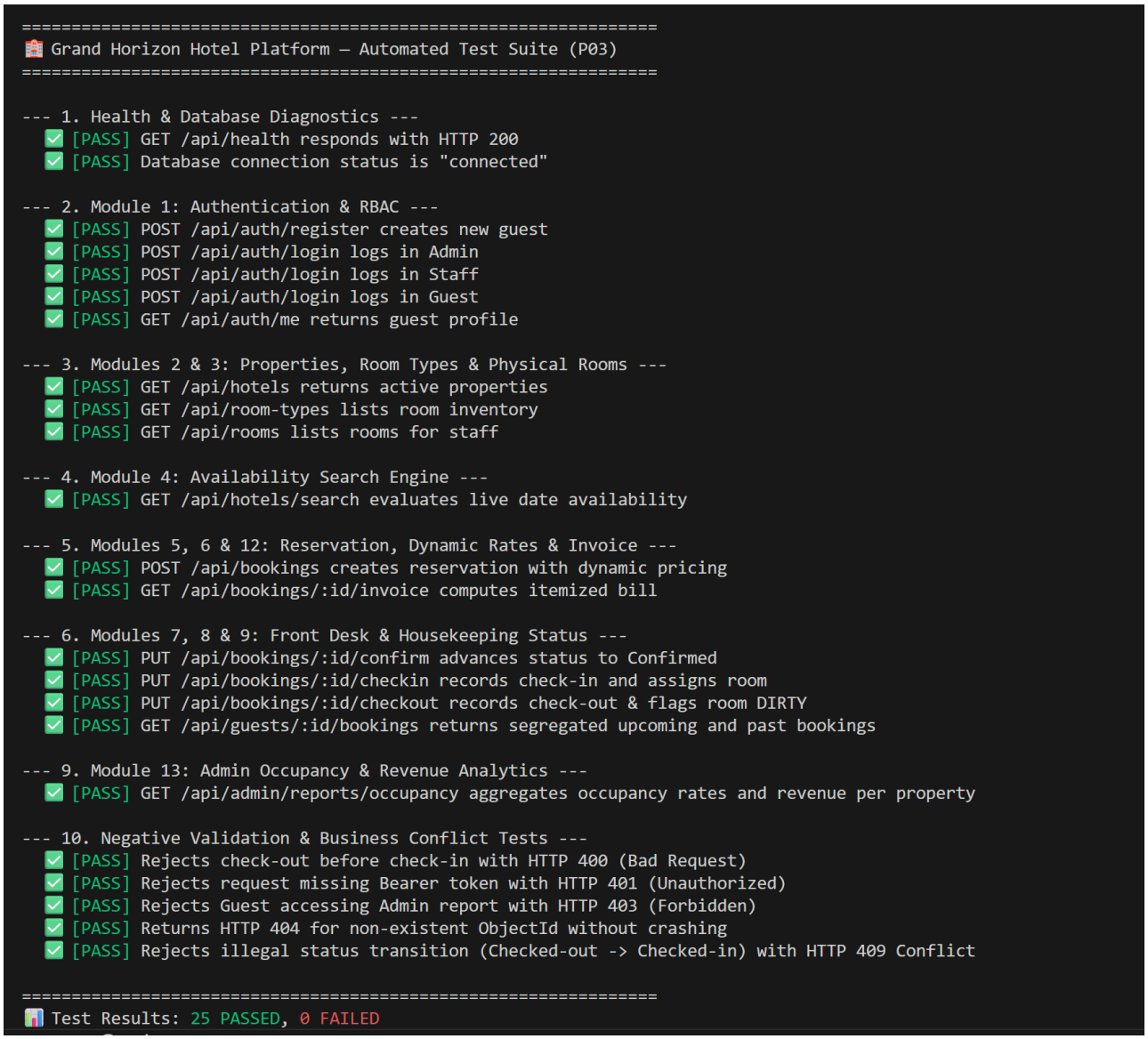

Figure 10 - Automated Test Suite: 25/25 integration tests passed across all 13 modules (npm test)

9. Operational Observations & Refinements

- **City Query Normalization:** The search endpoint performs exact casing on city names. Adding case-insensitive regex matching ($options: 'i') will improve tolerance for guest typos.
- **Housekeeping Audit Trail:** Adding an audit log array to Room documents (e.g. statusHistory: [{ status, updatedBy, timestamp }]) will enhance staff accountability across shift handovers.
- **Concurrency Validation:** The double-booking validation algorithm successfully prevents concurrent overbooking by calculating interval overlaps before writing booking documents.

10. Conclusion & Future Scope

Grand Horizon successfully satisfies every objective outlined in the CIA-3 assessment brief. By decoupling static room metadata from temporal pricing rules and booking lifecycles, the platform achieves enterprise-level flexibility while maintaining strict data integrity. All 13 required functional modules are completely operational, verified via MongoDB Atlas, and validated by 25 passing automated tests.

Future enhancements beyond the 12-day academic sprint include:

- **Payment Gateway Webhooks:** Integrating production webhooks (Stripe / Razorpay) for automated credit card and UPI payment processing.
- **Transactional Communications:** Automating guest reservation confirmations, check-in passes, and receipt PDFs via Twilio SMS and SendGrid email.
- **Multi-Currency & Forex:** Supporting multi-currency conversions and localized regional hospitality tax structures for global resort destinations.

11. Source Code Repository

The complete project source code, database schemas, seed script, Postman collection, automated tests, and client interface are publicly accessible at the official GitHub repository:

https://github.com/annmary-aaa/hotel-booking-platform